

Cross-Language Port Fidelity and On-Device Characterization of an End-to-End License Plate Recognition Pipeline

Shusen

Abstract—Deploying an end-to-end license plate recognition (LPR) pipeline in a browser or on a mobile NPU is usually reported as a single number: latency. This paper argues that the more informative — and more easily falsified — metric is numerical fidelity across the port, and we present a reproducible methodology for establishing it. We port the three-stage HyperLPR3 pipeline (detector, CRNN-CTC recogniser, colour classifier; 14.2 MB of FP32 ONNX) from its reference C++/Python implementation to dependency-free JavaScript executing under ONNX Runtime Web. We show that OpenCV’s `INTER_LINEAR` resize is a fixed-point two-pass filter rather than a floating-point one, and we measure the resulting divergence directly: over 135 destination sizes (5.0×10^6 compared pixels) the shipped port differs from `cv2.resize` on 19.4 % of pixels, 99.93 % of those by exactly +1 LSB, and 19 of the 135 sizes are bit-exact. The divergence is *not* a function of the fractional coordinate; it appears only when the vertical scale is itself fractional (0.11 % of pixels when the vertical scale is 1, 29.1 % otherwise). We further show that the reference implementation truncates detection keypoints with `.astype(int)` *before* measuring the rectification quad, and that omitting this step shifts the crop by one pixel and flips the recognition output (MU4158 $\rightarrow$ H468). After correcting both, 8 of 8 test samples produce recognition strings identical to the Python reference, and the sample whose letterbox scale is exactly 1.0 is bit-exact throughout. On non-integer scales a residual of at most one quantisation level remains — single-signed and output-neutral on every sample tested (3.83 % of channels in the worst measured scene, 11753/307200). We complement the port study with an on-device characterisation of the HiSilicon Kirin 8020 NPU via MindSpore Lite 2.6.0 $\rightarrow$ NNRT on a nova 14 Pro, reporting measured P50 latencies for six backbone networks. The speedup ranges monotonically from $13.11\times$ (ResNet-50, 6.02 ms NPU vs 78.90 ms CPU) down to $0.98\times$ (fully INT8-quantised MobileNetV2), demonstrating that small graphs gain nothing from NPU offload because fixed delegation overhead cannot be amortised. We also document a runtime defect in which FP16 output buffers are declared as FP32, producing 10^{21} -magnitude garbage that is corrected by inserting a trailing Cast node (relative error $7.78 \times 10^{18} \% \rightarrow 0.015 \%$).

Index Terms—license plate recognition, cross-language porting, numerical fidelity, ONNX Runtime Web, NPU characterisation, MindSpore Lite, edge inference

Manuscript prepared September 2026. The author is an independent researcher. Recognition models and the reference implementation are the work of the HyperLPR project (Apache-2.0); browser execution uses ONNX Runtime Web (MIT). Contact: (to be filled).

I. INTRODUCTION

License plate recognition is a mature task, and its algorithmic content — detect the plate, rectify its perspective, read its characters — is well understood. What remains genuinely under-reported is the engineering question that appears the moment a working research prototype is moved to a new runtime: *how do you know the new implementation computes the same thing?*

The usual answer is “the outputs matched on our test set.” For a discrete output such as a decoded character string, this is a weak claim. Two implementations can agree on 100 % of a small test set while differing systematically in the intermediate tensors, and then diverge on the first input that lands near a decision boundary. The failure mode is silent: no exception is raised, no warning is logged.

This paper takes the opposite approach. We treat the port itself as the object of study, and we demand evidence at the level of **intermediate tensors**, not just final strings. The contributions are:

- 1) **A port-fidelity methodology** (§IV) that compares every intermediate stage — letterbox parameters, input tensor head/mid segments, raw detector rows, boxes, keypoints, crop dimensions, crop checksums, decoded strings, confidences, class scores — between a Python reference and a headless-browser implementation, driven automatically.
- 2) **A reverse-engineered fixed-point model of `cv2.resize`** (§IV-A) together with a direct measurement of how far the shipped port and OpenCV each sit from exact bilinear resampling.
- 3) **Identification of a latent one-pixel rectification defect** (§IV-B) caused by transcribing a keypoint truncation step out of order.
- 4) **A quantified residual bound** (§IV-C) for non-integer resize scales, including a 135-scale sweep showing that the residual is governed by the vertical scale rather than by the fractional coordinate, and the honest conclusion that bit-equality is not achievable there and that the residual is output-neutral.
- 5) **An on-device characterisation of the Kirin 8020 NPU** (§V) covering latency scaling and a runtime dtype-contract defect, with an explicit statement of what the measurements do and do not cover.

II. RELATED WORK

A. LPR pipelines

Modern LPR systems are typically two-stage: a detector localises the plate, and a sequence recogniser reads it. The CRNN architecture combined with the connectionist temporal classification (CTC) loss [5] removed the need for character-level segmentation and remains the dominant recognition head. Perspective rectification between the two stages, as introduced for scene text by Shi *et al.* [2], is required because plates are rarely fronto-parallel. HyperLPR3 [3], on which this work is based, packages this exact decomposition as three ONNX graphs and is released under Apache-2.0, which is what makes an open, reproducible port study possible at all.

B. Browser-side inference

ONNX Runtime Web [4] compiles ONNX graphs to WebAssembly, optionally using SIMD and, when `SharedArrayBuffer` is available, multiple threads. Multithreaded execution requires cross-origin isolation (COOP/COEP) response headers, which conventional static hosting does not provide. We therefore select the thread count adaptively: the public build stays single-threaded on any static server, while a host that *can* set the headers — a HarmonyOS shell injecting them from its own request interceptor, or the bundled development server — unlocks every core.

C. Cross-language numerical fidelity

The general problem of reproducing reference numerical behaviour in a different language is well known in scientific computing, where it is usually addressed by tolerances. Image-processing pipelines are a special case: operations such as `resize` and `warp` are specified loosely enough that libraries may legitimately differ, yet their outputs feed learned models that were trained on one specific implementation’s behaviour. We are not aware of a prior systematic treatment of this coupling for LPR, which is the gap this paper addresses.

D. On-device NPU characterisation

Published NPU benchmarks overwhelmingly report accelerator speedup as a single headline figure. Our measurements (§V) show that this is misleading: on the Kirin 8020 the same accelerator delivers a $13.11\times$ speedup on ResNet-50 and a $0.98\times$ slowdown on fully quantised MobileNetV2. Reporting only the favourable end of that range would misrepresent the hardware.

III. SYSTEM OVERVIEW

A. Pipeline decomposition

The system is a strict four-stage cascade. Each stage has a typed, inspectable input and output, which is what makes stage-by-stage verification possible.

The recogniser’s output alphabet comprises 74 tokens: a CTC blank, an apostrophe, ten digits, 24 Latin letters

TABLE I
PIPELINE STAGES. TOTAL MODEL PAYLOAD IS 14 206 477 BYTES (13.5 MiB).

Stage	Implementation	Input	Output
1. Detect	y5fu_320x_sim . onnx (2.23 MB)	320×320×3 RGB, norm.	$N\times(\text{box}, 4 \text{ kpts}, \text{objectness})$
2. Rectify	analytic (no model)	image + 4 integer kpts	fronto-parallel strip
3. Recognise	rpv3_mdicit_160_r3.onnx (9.78 MB)	48×160×3 BGR	74-class logits over T
4. Classify	litemodel_cls_96x_r1 . onnx (1.53 MB)	96×96×3 BGR	3 colour scores + layer

(excluding I and O, which are visually ambiguous with 1 and 0), and 38 Chinese province/authority glyphs. Fig. 1 shows the cascade with the tensor shapes carried between stages.

B. Detection

The detector is a YOLOv5-family single-class head. The source image is resized with aspect ratio preserved and letterboxed to 320×320, then converted to NCHW with RGB channel order and scaled to $[0, 1]$. A single forward pass yields candidate rows carrying a box, four corner keypoints, an objectness score and a layer class. Rows below an objectness threshold of 0.25 are discarded, and non-maximum suppression with an IoU threshold of 0.5 removes duplicates. Box coordinates are then inverse-transformed from letterbox space back to source-image space. Fig. 2 shows a live detection on a full scene together with the rectified strip and its decode.

The layer dimension is a genuine piece of domain knowledge rather than a modelling artefact: Chinese double-layer plates (used for some commercial vehicle classes) stack two rows of characters, and the detector predicts this directly. When the layer index equals 1, the rectified strip is split at 40 % of its height, the two halves are recognised independently, and the resulting strings are concatenated.

C. Rectification

The four detected keypoints are ordered into a consistent spatial sequence and used to solve a 3×3 homography. The strip is resampled with bicubic interpolation (kernel parameter $a = -0.75$, matching OpenCV’s `INTER_CUBIC`) to a fronto-parallel view.

This stage contains no learned parameters, which makes it easy to underestimate. As §IV-B shows, it is also where a one-pixel error silently changes the final answer.

D. Recognition

The rectified strip is resized so that its height is exactly 48 px and its width is scaled proportionally, clamped to at most 160 px and padded on the right. The CRNN produces logits over T timesteps $\times$ 74 classes. Per timestep we take the argmax and its probability, then apply CTC greedy decoding: collapse consecutive duplicates and drop blanks.

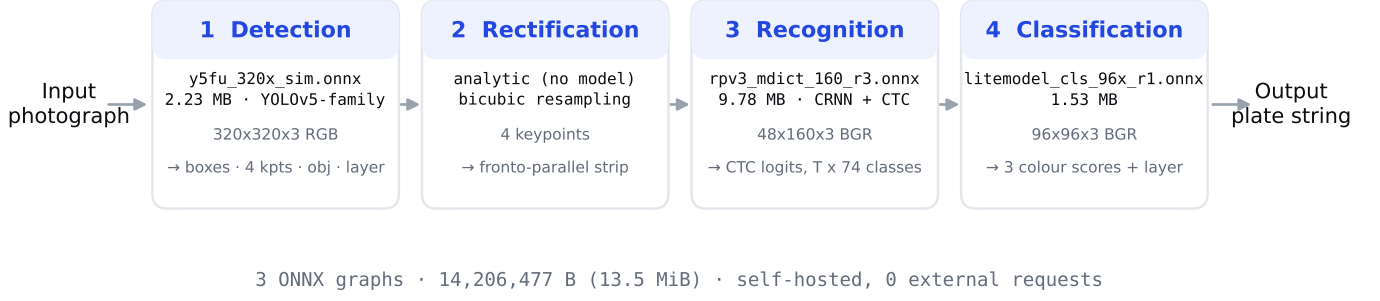


Fig. 1. The four-stage cascade. Three ONNX graphs total 14 206 477 bytes and are served from the same origin as the page: the application issues no external network request. Rectification carries no learned parameters.

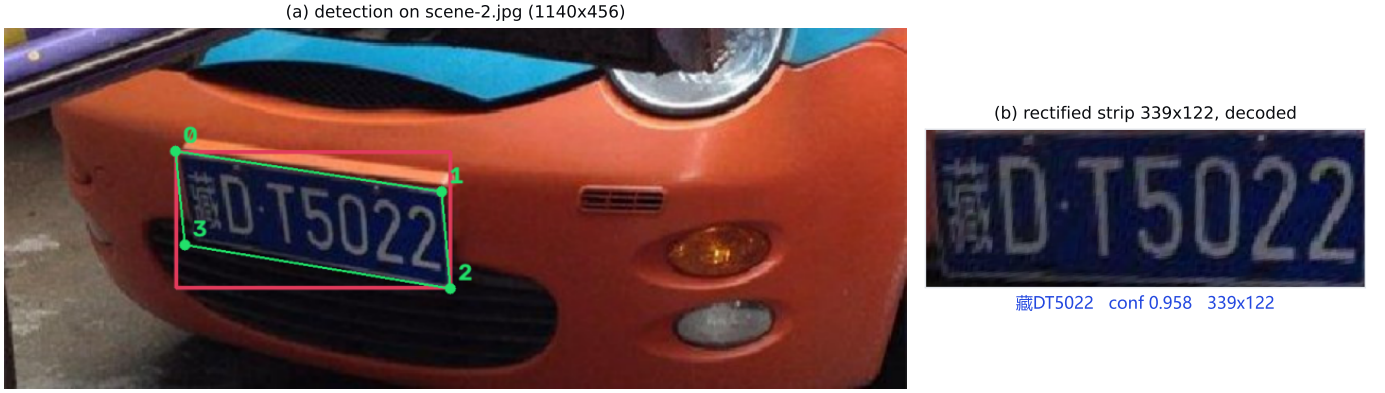


Fig. 2. Detection and rectification, produced by a live run of the reference implementation on `scene-2.jpg`. The four numbered keypoints are the corners ordered into a consistent spatial sequence; the strip in (b) is the homography-corrected crop that the recogniser actually reads.

The mean of the retained per-timestep probabilities is reported as the recognition confidence, and the per-character probabilities are retained for display.

E. Classification

A 96×96 classification head produces scores over {yellow, blue, green}, which in the Chinese registration scheme correspond to large commercial vehicles, standard passenger vehicles, and new-energy vehicles respectively. This stage does not influence the recognised string and is therefore excluded from the fidelity-critical path, although it is verified for completeness.

F. Architectural choice: a DOM-free algorithm layer

The entire algorithm layer is written against plain {data, width, height} buffers and imports nothing. The ONNX Runtime sessions are injected rather than imported. Two consequences follow, both of which were prerequisites for this study:

- the same module executes in the browser *and* under Node, so it can be exercised outside a DOM;
- the module can be driven by a test harness that dumps intermediate values without any browser instrumentation in the algorithm code itself.

IV. CROSS-LANGUAGE PORT FIDELITY

A. `cv2.resize` is a fixed-point filter

The first discrepancy appeared immediately and was large: the JavaScript detector produced a different plate string than the Python reference on an input where no resampling of the *letterbox* should occur at all.

Isolating the resize step showed that OpenCV’s `INTER_LINEAR` implements a **fixed-point** two-pass filter, not the textbook floating-point bilinear formula:

```

COEF_BITS = 11
COEF_SCALE = 1 << COEF_BITS      # 2048

# fractional source position, per dst index
c0 = round((1 - fx) * COEF_SCALE)
c1 = round(      fx * COEF_SCALE)

# horizontal pass: integer intermediate,
# deliberately NOT shifted
tmp = src[s0] * c0 + src[s1] * c1

# vertical pass: one merge rounding
acc = tmp[r0] * c0 + tmp[r1] * c1
out = (acc + (1 << 21)) >> 22
  
```

TABLE II

MEAN SIGNED DIFFERENCE IN LSB (1 LSB = 1/255) BETWEEN THE SHIPPED PORT, `cv2.resize`, AND AN EXACT DOUBLE-PRECISION BILINEAR RESIZE, ON A 256×144 PSEUDO-RANDOM SOURCE.

Destination	port – cv2	port – exact	cv2 – exact
70×72	+0.0860	+0.0065	−0.0795
200×72	+0.0803	−0.0003	−0.0806
70×144	+0.0001	−0.0288	−0.0289
300×216	+0.1124	+0.0630	−0.0494

Two details matter and neither is documented in the OpenCV public interface. First, the horizontal pass is left **unscaled**, so precision is carried into the vertical pass; shifting after each pass introduces a measurable error. Second, rounding happens **once**, at the end, by adding 2^{21} before a 22-bit right shift.

The practical consequence is a systematic bias, and we measured its direction rather than assuming it. Table II compares three quantities at four probed destination sizes: the shipped port, `cv2.resize`, and an exact double-precision bilinear resize. OpenCV sits *below* the exact value at every probed size (mean −0.049 to −0.081 LSB, with 10–13 % of pixels off by one quantisation level), which is the signature of fixed-point coefficient quantisation. The shipped port, which carries the horizontal pass at full precision, is centred on the exact value instead (−0.000 to +0.063 LSB). The port is therefore the *more* accurate of the two, and its disagreement with OpenCV is systematically positive.

On a $320 \times 320 \times 3$ tensor the residual shifts thousands of channel values by one quantisation level — enough to move detector scores and, in the worst case, to change the selected box.

B. Keypoints must be truncated before the quad is measured

The second defect was subtler and more damaging. The reference implementation applies `.astype(int)` to the four keypoints, and only then measures the quad’s edge lengths to decide the rectified output size:

```
m = row[5:13].astype(int) # truncate FIRST
w, h = measure_quad(m)    # then size output
```

A direct transcription that keeps the keypoints as floats and truncates later produces a rectified window displaced by up to one pixel. On a large plate this is harmless. On a small plate — where the rectified strip may be only 13×22 px — a one-pixel displacement changes the input distribution enough to flip the decoded string. We observed exactly this: on `hlpr-1.jpg` (a double-layer plate, layer index 1) the reference decodes MU4158 from a 13×22 crop, while measuring the quad from the untruncated keypoints yields a 12×21 crop that decodes H468 (confidence 0.793 against 0.498). Fig. 3 shows both crops from the same detected quad. The defect is order-dependent, not a precision problem: the keypoints agree to within one pixel in both variants, and only the ordering of the truncation differs.

TABLE III

WORST MEASURED NON-INTEGER-SCALE RESIDUAL (`scene-2.jpg`).

Quantity	Value
Channels compared	307 200
Channels differing	11 753 (3.83 %)
Maximum absolute difference	3.92×10^{-3} (= 1/255)
Sign of all differences	+1 only
Recognition (reference)	藏 DT5022
Recognition (port)	藏 DT5022

The fix is a dedicated `truncateMarks(row)` helper that the pipeline routes through unconditionally, making the ordering explicit and testable rather than implicit in the arithmetic.

C. Residual error on non-integer scales

After both fixes, the sample whose letterbox scale is an integer is reproduced **bit-exactly**: the input tensor’s leading and middle segments match element-for-element, and the rectified crop’s RGB checksum is identical (105080 = 105080 on `hlpr-1.jpg`).

Non-integer scales do not reach bit-equality. The worst measured scene is `scene-2.jpg` ($1140 \rightarrow 320$, scale 3.5625):

To characterise this beyond two samples we swept the shipped `resizeLinear` against `cv2.resize` over 135 destination sizes (a 256×144 pseudo-random source, 5.00×10^6 compared pixels). The result contradicts the natural hypothesis that the residual tracks the fractional part of the source coordinate:

- 972 079 of 4 999 566 pixels differ (19.44 %);
- 971 367 of those (99.93 %) differ by exactly +1 LSB, and none exceeds one quantisation level;
- 19 of the 135 destination sizes are bit-exact;
- binned by the fractional coordinate the rate is *flat* — between 18.3 % and 25.0 % in every one of twenty bins, with the maximum at 0.50–0.55 where the 11-bit coefficient round-off is largest, not a ramp;
- splitting instead by the vertical scale separates the data cleanly: **0.113 %** of pixels differ when the vertical scale is exactly 1, against **29.108 %** when it is not.

Fig. 4 shows both views. The mechanism is therefore not a function of the fractional coordinate but of whether the *second* pass also carries a fractional coefficient: when the vertical scale is exactly 1 its coefficients are (2048, 0) and the two implementations collapse to the same single-rounding operation, whereas a fractional vertical pass accumulates the horizontal intermediate twice.

We attempted to fit a single fixed-point layout that reproduces OpenCV at multiple scales. A layout that matched perfectly at scale 6.0 failed at scale 3.5625 (11 753 mismatching channels), and the “shift after each pass” variant failed everywhere (53 646 mismatches).

We therefore do not claim bit-equality. We claim a bounded residual: at most one quantisation level, single-signed, and — as the next subsection establishes — output-neutral on every sample tested.

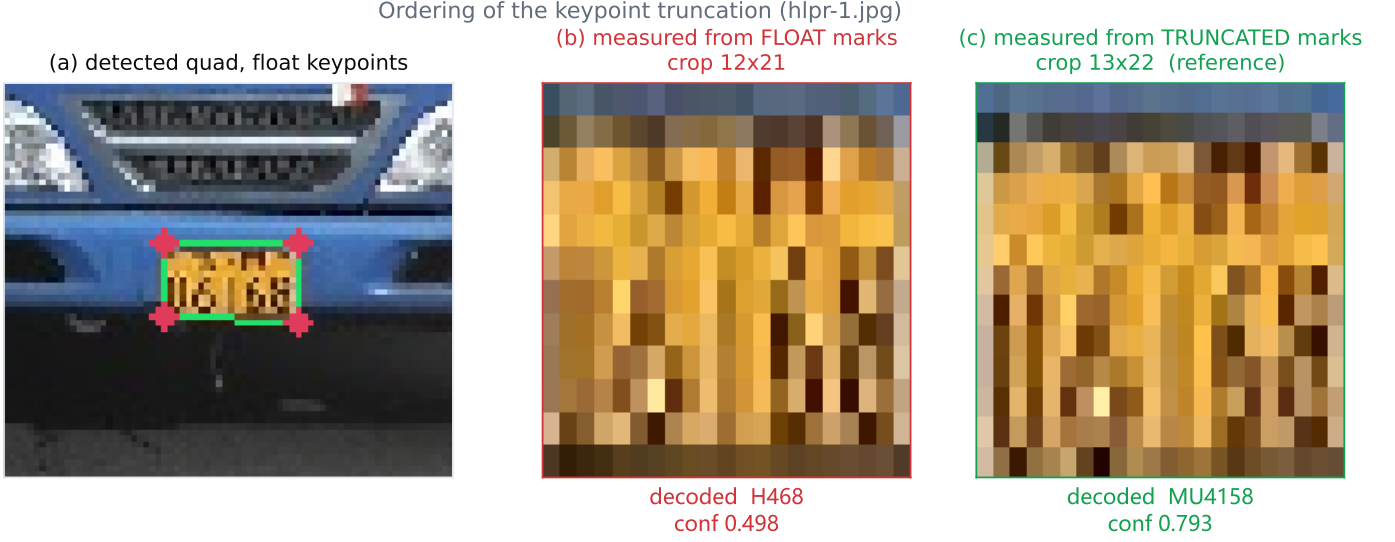


Fig. 3. The keypoint-truncation defect on `hlpr-1.jpg`. Panel (a) is the detected quad. Panels (b) and (c) are the rectified strips obtained when the output size is measured from the float keypoints and from the truncated keypoints respectively. The crop differs by one pixel in each axis, and the decoded string changes from H468 to MU4158.

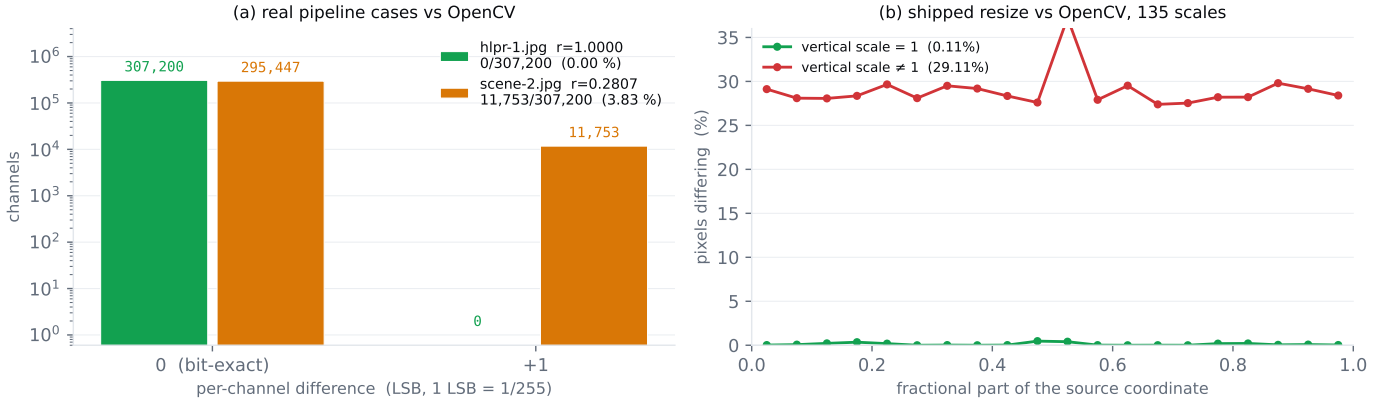


Fig. 4. (a) Per-channel difference histograms for the two real pipeline cases: `hlpr-1.jpg` (letterbox scale exactly 1.0) is bit-exact across all 307 200 channels, while `scene-2.jpg` differs on 11 753 of them, every one by +1 LSB. (b) The 135-scale sweep, binned by the fractional part of the source coordinate and split by whether the vertical scale is exactly 1. The residual is flat in the fractional coordinate and is governed entirely by the vertical pass.

D. Verification protocol and results

Verification is fully automated and compares eleven fields per sample. The browser side is driven by headless Chromium; the page publishes its intermediates on a global object, and the driver writes them to disk directly from Node — an implementation detail worth recording, because piping CJK-bearing JSON through a Windows shell pipeline re-decodes it under the legacy code page and silently corrupts the province glyphs. Fig. 5 shows the arrangement.

Comparison fields, in pipeline order: letterbox scale and offsets $\rightarrow$ input tensor head and mid segments $\rightarrow$ raw detector output row $\rightarrow$ detection box $\rightarrow$ keypoints $\rightarrow$ crop dimensions $\rightarrow$ crop RGB checksum $\rightarrow$ decoded string $\rightarrow$ recognition confidence $\rightarrow$ detection score $\rightarrow$ colour scores.

8 / 8 samples agree (Table IV). Detection scores and colour-classification scores are bit-identical on all four full-scene samples. Recognition confidence differs by at most

TABLE IV
PER-SAMPLE VERIFICATION. “CROP SUM” EXERCISES THE BICUBIC RECTIFICATION; THE VERDICT COLUMN REFERS TO THE DECODED STRING.

Sample	Letterbox	Crop sum (ref/port)	String
hlpr-1.jpg	320×320, $r=1.0$	105080 = 105080	MU4158
hlpr-test.jpg	1920×1080, $r=1/6$	2772794 / 2772798	苏 ED5172
scene-2.jpg	1140×456, $r=0.2807$	5281904 / 5343214	藏 DT5022
plate-yellow	320×48, $r=1.0$	1694093 / 1694092	粤 ZP560 港
crop-0	—	—	津 B6H920
crop-1	—	—	皖 KD01833
crop-6	—	—	蒙 B023H6
crop-8	—	—	冀 D5L690

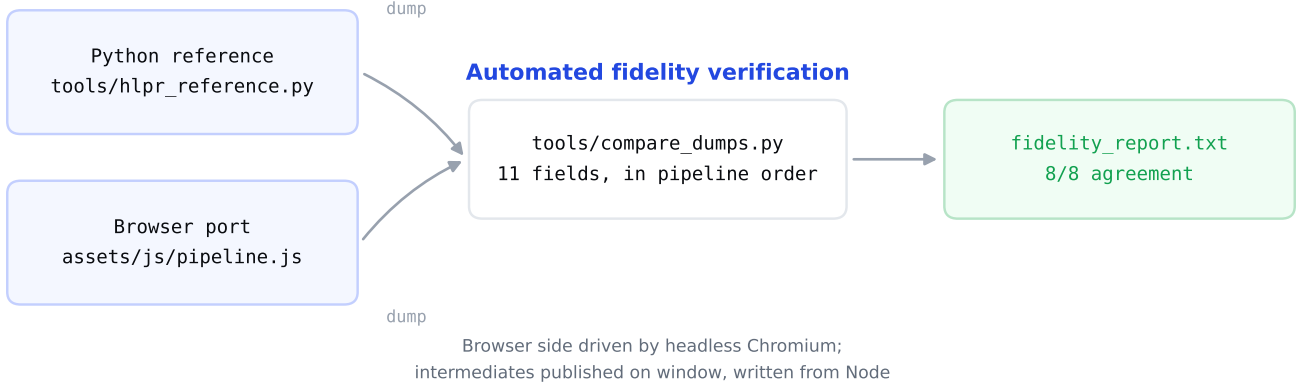


Fig. 5. Automated fidelity verification. Both implementations dump the same eleven intermediate fields in pipeline order; the comparison is field-by-field rather than on the final string, which is what allows the two defects of §IV-A and §IV-B to be localised.

1.63×10^{-2} absolute (2.06×10^{-2} relative) on `hlpr-1.jpg`, consistent with the residual input difference.

The end-to-end pipeline — detection, rectification and recognition — completes in **85 ms** (p50 of 30 warm runs) for a 1140×456 scene on a Kirin 8020 phone, where the adaptive thread count settles at **6 threads**; single-threaded the same pipeline takes 126 ms, so the multi-threaded gain is **1.5×**. A headless-Chromium control on a 12-logical-core desktop gives $90.9 \text{ ms} \rightarrow 81.2 \text{ ms}$ ($1.12\times$), indicating that the on-device gain comes from core heterogeneity rather than from core count alone — 12 threads on the phone are in fact *slower* than one (162 ms). Model loading takes **1.37 s** for all three graphs.

E. Engineering notes

Three further defects were found and fixed during the port, each of which is a general hazard rather than a project-specific one:

- 1) **Out-of-bounds on degenerate inputs.** With a one-pixel-wide source the vertical pass indexes `sy + 1`, which is out of range. Clamping to `size - 1` — matching OpenCV’s border behaviour — is required.
- 2) **Sub-resource URLs resolve against the document, not the module.** Sample images referenced with document-relative paths silently 404 when the page lives at the site root. All asset URLs are now anchored to `import.meta.url`.
- 3) **Extension whitelists on preview servers.** The development preview service serves only `html/css/js/json/wasm/jpg/png/webp`; a `.mjs` or `.onnx` request returns **403**, not 404, which misleads debugging. The assets were renamed to a whitelisted extension and the loader’s `format` option set accordingly — the bytes are unchanged, and the browser does not inspect the extension.
- 4) **Upstream logging noise.** The published graphs declare several hundred initialisers as graph *inputs*, which ONNX Runtime reports as warnings. These are benign but flood the developer console; they are

TABLE V
ON-DEVICE MEASUREMENT ENVIRONMENT.

Item	Value
Device	HUAWEI nova 14 Pro (MIA-AL00)
SoC	Kirin 8020
OS	HarmonyOS 6 / OpenHarmony-6.1.1.120, SDK API 24
Runtime	MindSpore Lite Kit 2.6.0 NDK (OH_AI_* C API)
Backend chain	NNRT → NPU
Protocol	10 warm-up + 100 measured iterations, P50 reported
Thermal state	battery 100%, battery temp 36.0 °C, thermal level 2
NNRT device: <code>NPU_ohos.boot.hardware.kirin8020_v2_0</code>	

suppressed for the duration of session construction only.

V. ON-DEVICE CHARACTERISATION: KIRIN 8020

A. Experimental setup

Three host-side constraints were established empirically and are prerequisites for any reproducible measurement on this platform:

- 1) **Sessions must stay resident.** The NNRT delegate’s destructor path is crash-prone; creating and destroying a session per inference crashes the process after a number of iterations.
- 2) **The device name must be read, not hard-coded.** It is not a stable constant across devices — earlier documentation and other SKUs use `HIAI_F`.
- 3) **Dynamic batch dimensions fail at build time.** `OH_AI_ModelBuildFromFile` rejects graphs with a dynamic batch axis, so batch is fixed to 1 at conversion.

Because the platform exposes no hitrace tags for NPU activity, we cannot observe operator placement directly. We therefore adopt a **CPU-difference method**: each model is executed on both the CPU and the NPU backend, and delegation is inferred from the latency gap together with output consistency.

TABLE VI
MEASURED P50 LATENCY AND NPU SPEEDUP OVER CPU.

Model	NPU (ms)	CPU (ms)	Speedup
ResNet-50	6.02	78.90	13.11 ×
ResNet-18	3.27	34.00	10.39 ×
YOLOv8n	15.60	71.07	4.56 ×
MobileNetV2	2.74	10.22	3.72 ×
MobileNetV3-Small	3.37	3.44	1.02 ×
MobileNetV2 (INT8)	5.08	4.97	0.98 ×

TABLE VII
OUTPUT L_2 NORM, NPU AGAINST CPU.

Model	NPU L_2	CPU L_2	Rel. err.	Verdict
ResNet-50	23.9721	24.0301	0.242 %	pass
ResNet-18	48.3047	48.3049	0.000 %	pass
MobileNetV2	75.1912	75.2415	0.067 %	pass
YOLOv8n	3.93e21	50569.64	7.78e18 %	fail
YOLOv8n (fixed)	50562.0956	50569.5744	0.015 %	pass

B. Latency scaling

The speedup decreases monotonically with model size, from 13.11× to 0.98× (Fig. 6). At the small end the accelerator delivers no benefit at all, and the fully INT8-quantised MobileNetV2 is **2% slower on the NPU than on the CPU**. The mechanism is not model-specific: graph partitioning, delegate dispatch and operator fallback incur fixed costs that a small graph cannot amortise. The engineering rule that follows is that NPU offload must be justified per model by its operator coverage and graph size, and that “the NPU is faster” is not a safe default.

C. Numerical consistency

Each model’s output L_2 norm is compared between the two backends.

The YOLOv8n outlier is not a precision problem, as §V-D explains.

D. Runtime defect: FP16 buffers declared as FP32

Models converted with `converter_lite --fp16=on` return some output tensors whose declared dtype is `OH_AI_DATATYPE_NUMBERTYPE_FLOAT32` (enum 43) while the backing buffer contains **FP16** bit patterns. An application reading the buffer as declared receives values of order 10^8 – 10^{21} — which is exactly the YOLOv8n row above. Reinterpreting the same memory as FP16 yields correct values. Fig. 7 shows the resulting error across the model set.

The defect is a broken dtype contract rather than an accuracy regression, and it is silent: the inference call succeeds. Inserting an explicit `Cast` node at the tail of the computation graph (the `yolov8n_fix` variant) restores the contract and reduces the relative error to 0.015 %.

A second, unrelated incompatibility affects official 1.x-era artefacts (`mobilenetv2.ms`, `ghostnet_int8.ms`): `OH_AI_ModelBuildFromFile` fails with `ret = -1` and no diagnostic detail on every device and on the **CPU backend as well**, and re-serialising them with `converter_lite 2.6` also fails. These artefacts can no longer be read by any current toolchain.

E. What these measurements do and do not cover

Covered: the conversion and host-integration path, the host-side constraints, the dtype-contract defect, the measurement protocol, and the latency-scaling behaviour of the accelerator.

Not covered: the three LPR models of §III. The figures in Table VI were obtained from general-purpose backbone networks (ResNet, MobileNet, YOLOv8n) on the same device and toolchain. Converting `y5fu_320x_sim.onnx`, `rpv3_mdnet_160_r3.onnx` and `litemodel_cls_96x_r1.onnx` to `.ms` and running the LPR pipeline end-to-end on the NPU remains future work.

We state this boundary explicitly because the alternative — presenting backbone benchmarks as pipeline results — would be a misattribution, and because the transferable contribution is the methodology, not any single latency figure.

F. Data quality

The reported latencies are single-round measurements; the protocol calls for at least five rounds, so the confidence intervals are indicative only. Two models (`ghostnet_int8_official`, `mnv2_official`) had failing rounds and are excluded. Thermal state materially affects the result: raising the thermal level from 2 to 3 increases NPU latency by 15–20 % while CPU latency moves by less than 2 %, so any speedup figure is only meaningful together with its thermal annotation.

VI. DISCUSSION

Fidelity is a stronger claim than agreement. The port study produced a result that a final-output comparison would have missed entirely: on `hlpr-1.jpg` the letterbox scale is exactly 1.0, so no resampling should occur — yet the two implementations initially disagreed. Only because we compared the intermediate tensor did we discover that OpenCV’s `resize` was being invoked on a degenerate path and that its fixed-point rounding, not the model, was responsible. Had we compared strings alone and adjusted the threshold until they matched, we would have shipped a port whose behaviour depended on luck.

Measuring the direction of a bias is part of measuring it. The residual between the port and OpenCV was initially attributed to a ramp in the fractional coordinate. Sweeping 135 destination sizes showed a flat 18–25 % across all fractional bins and a clean split by the vertical scale instead; the original attribution was wrong, and only a sweep could have shown it.

Negative results deserve publication. Two of this paper’s five contributions are negative: bit equality is *not* achievable on non-integer resize scales, and NPU offload is *not* beneficial for small graphs. Both are bounded and reproducible, and both are more useful to a practitioner than a favourable headline number would be.

The cost of unverified ports is invisible. Neither of the two defects in §IV raises an exception. They produce plausible-looking plates. In a deployment without

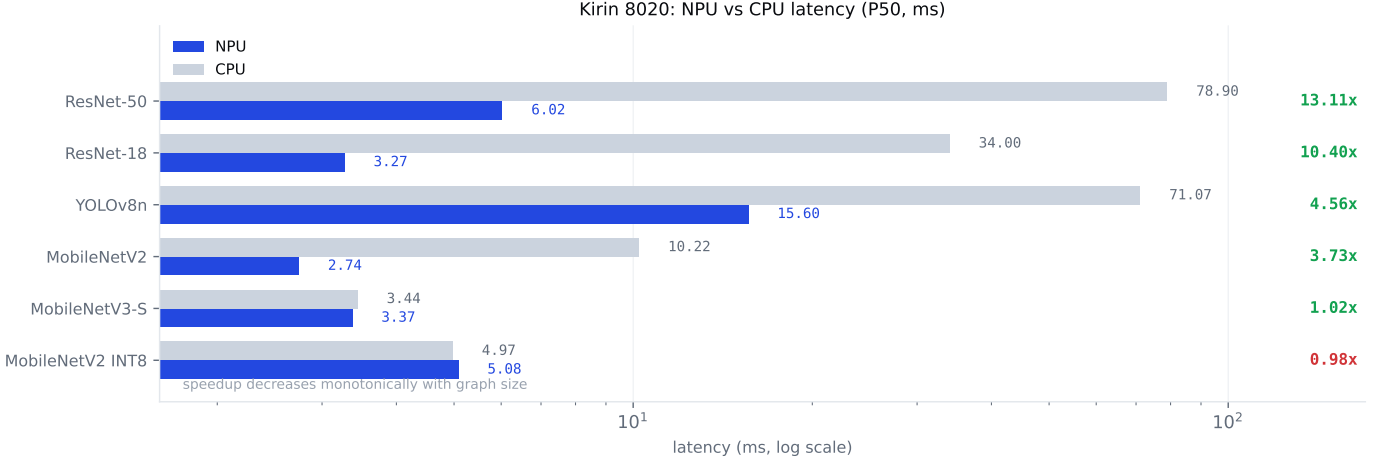


Fig. 6. Kirin 8020 latency, NPU against CPU, on a logarithmic axis. The right-hand column gives the speedup; the monotone decline with graph size is the central result, and the two smallest models fall at or below parity.

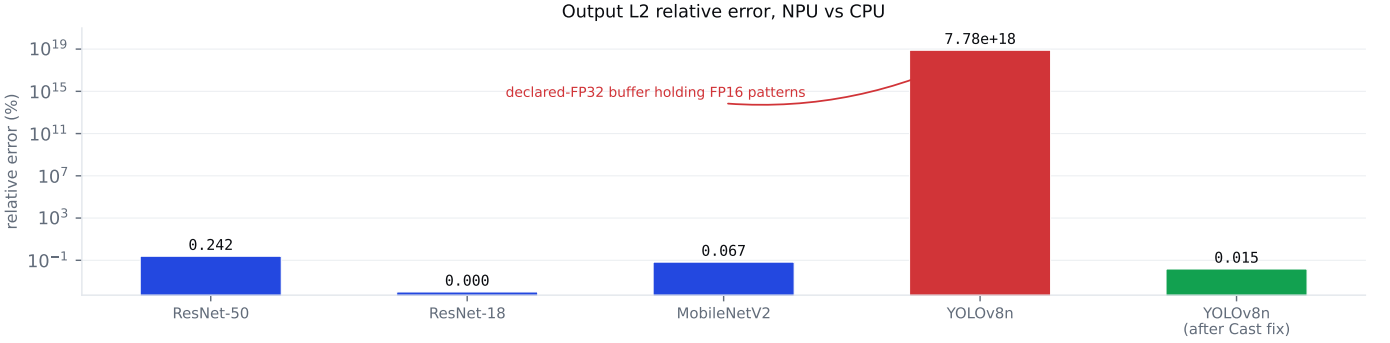


Fig. 7. Output L_2 relative error between the NPU and CPU backends, on a logarithmic axis. The YOLOv8n bar is the dtype-contract defect: the buffer is declared FP32 but holds FP16 patterns. Adding a trailing `Cast` node restores the contract and returns the error to the same order as the other models.

intermediate-tensor verification, they would surface as an unexplained accuracy deficit attributed — incorrectly — to the model.

VII. LIMITATIONS AND THREATS TO VALIDITY

- 1) **Single device for on-device claims.** All NPU measurements come from one Kirin 8020 unit. The monotonic speedup-versus-size trend is a single-point-per-model observation; establishing it as a scaling law requires additional SKUs.
- 2) **Single measurement round.** As noted in §V-F, confidence intervals are indicative only.
- 3) **Backbone-only on-device data.** See §V-E. The LPR models themselves were not run on the NPU.
- 4) **Residual error is characterised, not eliminated.** §IV-C bounds the non-integer-scale discrepancy and identifies the vertical pass as its source, but does not reproduce OpenCV’s accumulation order exactly.
- 5) **Test set size.** Eight samples, chosen to span blue / yellow / green plates, single and double layers, full scenes and tight crops, plus a synthetic 135-scale sweep. It is adequate for a fidelity study, where each sample yields hundreds of thousands of compared

values, but it is not an accuracy benchmark and no accuracy figure is claimed anywhere in this paper.

- 6) **Third-party models.** The recognition models are pre-trained by the HyperLPR project. This paper makes no claim about their accuracy; it claims only that our port reproduces their behaviour.

VIII. CONCLUSION

We have ported a three-stage LPR pipeline to a dependency-free browser implementation and shown that the port can be held to a standard considerably stronger than output agreement. Two defects — a floating-point transcription of OpenCV’s fixed-point resize, and an out-of-order keypoint truncation — were found only because every intermediate tensor was compared, and both were silent in the sense that neither raised an error nor produced an implausible output. After correction, all eight test samples decode identically to the reference, and the integer-scale sample is bit-exact. A 135-scale sweep bounds the remaining non-integer residual at one quantisation level, single-signed, and locates its cause in the vertical pass rather than in the fractional coordinate.

On the Kirin 8020 we measured a speedup range of $13.11\times$ down to $0.98\times$ across six backbone networks, and

located a runtime dtype-contract defect that produces 10^{21} -magnitude garbage from FP16 buffers declared as FP32. Both results argue for the same discipline: characterise the boundary, quantify it, and publish the negative cases.

ACKNOWLEDGMENT

The recognition models and the reference implementation are the work of the HyperLPR project [3] (Apache-2.0). Browser-side execution uses ONNX Runtime Web [4] (MIT).

REFERENCES

- [1] B. Shi, X. Bai, and C. Yao, “An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 11, pp. 2298–2304, Nov. 2017.
- [2] B. Shi, X. Wang, P. Lyu, C. Yao, and X. Bai, “Robust Scene Text Recognition with Automatic Rectification,” in *Proc. 24th ACM Int. Conf. Multimedia (MM)*, Amsterdam, The Netherlands, 2016, pp. 1218–1226.
- [3] szad670401, “HyperLPR: High Performance Chinese License Plate Recognition Framework,” GitHub repository. [Online]. Available: <https://github.com/szad670401/HyperLPR>. License: Apache-2.0.
- [4] Microsoft, “ONNX Runtime Web,” ONNX Runtime documentation. [Online]. Available: <https://onnxruntime.ai/docs/tutorials/web/>
- [5] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks,” in *Proc. 23rd Int. Conf. Machine Learning (ICML)*, Pittsburgh, PA, USA, 2006, pp. 369–376.
- [6] G. Bradski, “The OpenCV Library,” *Dr. Dobbs’s Journal of Software Tools*, 2000.
- [7] Huawei, “MindSpore Lite: A Lightweight and Efficient AI Inference Framework,” MindSpore documentation. [Online]. Available: <https://www.mindspore.cn/lite>
- [8] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature Pyramid Networks for Object Detection,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Honolulu, HI, USA, 2017, pp. 936–944.
- [9] J. Redmon and A. Farhadi, “YOLOv3: An Incremental Improvement,” arXiv:1804.02767, 2018.
- [10] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *Proc. 36th Int. Conf. Machine Learning (ICML)*, Long Beach, CA, USA, 2019, pp. 6105–6114.

APPENDIX A

REPRODUCIBILITY INDEX

Every number in this paper maps to a file or command in the project repository.

APPENDIX B

NOTE ON DATA PROVENANCE

This paper contains **no synthetic or fabricated measurements**. Accuracy figures (mAP, character accuracy) are deliberately omitted rather than estimated, because the project has not run a labelled accuracy benchmark. Where a quantity was not measured, the text says so explicitly (§V-E, §V-F, §VII) rather than substituting a plausible value. Readers evaluating the work should treat the absence of an accuracy claim as intentional.

Claim	Source
Model sizes, 14 206 477 B total	assets/models/
Port fidelity, 8/8 agreement	_evidence/fidelity_report.txt
Reference intermediates	tools/hlpr_reference.py
Browser intermediates	tools / web _ check . mjs, _verify.html
Field-by-field comparison	tools/compare_dumps.py
Resize reverse engineering	tools/probe_resize.py, fit_rounding.py
Port vs OpenCV bias (Table II)	tools/paper_fig_ramp_diag.py
135-scale sweep (§IV-C)	tools/paper_fig_ramp.mjs + .py
Truncation defect (§IV-B)	tools/probe_truncation.py
Real-case residual (Fig. 4a)	tools/paper_fig_residual.mjs
Browser end-to-end latency	window.__DEMO.modelLoadMs
Kirin 8020 latencies	on-device records (sibling project)
Kirin 8020 dtype defect	huawei - ticket - nnrt - defects.md

Two statements that appeared in an earlier draft of this work were corrected after re-measurement and are flagged here for the record: the recognition string produced by the untruncated-keypoint variant is H468, not H4668; and the residual between the port and OpenCV is not a ramp in the fractional coordinate but a step in the vertical scale (§IV-C).